

3 — Module: state created once, shared by every importer

IMPORTS SCOPE SINGLETON

Q. Two files both import { current } from session.js. Do they each get their own currentUser?

ANSWER. No. A module's top-level code runs exactly once, the first time anything imports it. So `currentUser` is created once, and every file that imports `session.js` reaches that single shared value, never a private copy.

WHY IT WORKS. The variable is never exported, so it is private to the module; the only way in or out is through the exported `login` and `current` functions. That makes the module a closure stretched over a whole file. One shared instance, touched only through its public functions, lives for the whole program. The pattern has a name: a singleton.

```
let currentUser = "John";           // private
module state

export function login(name) {        // writes
  currentUser
  currentUser = name;
}

export function current() {          // reads
  currentUser
  return currentUser;
}
```

examples/session.js

```
import { login, current } from "../session.js";

console.log(current());              // "John"
// (the initial value)
login("Ada Lovelace");               // changes
// the shared value
console.log(current());              // "Ada
// Lovelace" (seen everywhere)
```

examples/app.js

ONE SHARED STORE

session.js

created ONCE

```
let currentUser = "John"
```

login()

current()

THE ONLY DOORS

↓ **import**

app.js reaches the SAME value

- ① **Summary.** A module runs once, so any top-level variable becomes a shared singleton, reached only through the functions it exports. The value is shared now, but reads and writes are still plain — what if a *read* itself could run code?